Precision as a software choice: FP64, double-double, triple-double and beyond from the INT8 tensor cores of a laptop GPU

Shaban Ejupi
Independent researcher
ORCID 0009-0008-9406-0042

3 November 2026

Abstract

Consumer and laptop GPUs run FP64 at 1/64 of their FP32 rate: on a laptop Blackwell GPU (RTX PRO 3000, sm_120) we measure 232 TOPS of INT8 `mma.sync` against 0.449 TFLOPS of native FP64 (517×). With the Ozaki scheme II we implement in CUDA C and PTX matrix multiplication at FP64 (6.7 TFLOPS at $n = 4096$, where cuBLAS 13's own emulation reaches 8.5), double-double (3.81 TFLOPS), triple-double (2.03 TFLOPS) and up to 424 bits from the same INT8 units, to our knowledge the first such GEMMs beyond FP64 on GPU INT8 tensor cores. Solving $Ax = b$ with $\kappa(A) = 10^{20}$ in double-double takes 0.692 s at $n = 8192$, 22× sooner than the same LU blocked on 24 CPU threads and 629× sooner than `__float128`; at $\kappa = 10^{36}$, where double-double has no correct digit, a triple-double LU is 9× faster than the same blocked CPU LU and 166× than MPFR. Exact integer matrix products with 1 015- to 131 063-bit entries run 10–19× faster than FLINT on 24 CPU threads; exact determinants come up to 355× (int8, $n = 8192$) and 404× (64-bit entries, $n = 4096$) sooner than FLINT; on 15 of NIST Matrix Market's most ill-conditioned real matrices the certified exact solution is 9–7 857× faster than SPEX where SPEX finished. Power-bound at 94–117 W, the GPU reaches 91% of its sustained INT8 rate with Strassen over $\mathbb{Z}/m$ (in classical operations), and an FP64-accurate DGEMM costs 13.9× less energy than its native FP64, each at its most efficient clock. The same units give correctly rounded MXFP4, NVFP4, FP8 and BF16 products: every output of Kimi K3's 48 expert matrices (2.78 trillion parameters) is certified, where BF16 tensor cores with FP32 accumulation round 51–83% of them differently; Nemotron 3 Ultra (560 billion parameters) runs from disk with every linear map correctly rounded at 19.5–19.7 s per token, as fast as FP32 accumulation.

Përmbledhje (shqip). Kartelat grafike të laptopëve e ekzekutojnë FP64 me 1/64 e shpejtësisë FP32, ndërsa tensor core-t INT8 janë qindra herë më të shpejtë. Mbi një GPU Blackwell laptopi matëm 232 TOPS INT8 kundrejt 0,449 TFLOPS FP64 (517×). Me skemën Ozaki II (prodhime të sakta të mbetjeve dhe teorema kineze e mbetjeve) ndërtuam shumëzim matricash me saktësi FP64 (6,7 TFLOPS; emulimi i vetë cuBLAS 13 arrin 8,5), double-double, triple-double dhe arbitrare (deri 424 bit) nga të njëjtat njësi INT8. Te një punë reale—$Ax = b$ me $\kappa = 10^{20}$—zgjidhja double-double mbaron 22× më shpejt se e njëjta LU e bllokuar mbi 24 fije CPU dhe 629× më shpejt se LU `__float128` ($n = 8192$), me gabim po aq të vogël, dhe me ∼270× më pak energji se `__float128` ($n = 4096$). Me prime nën 2^{16} e 2^{23} të ndara në bajte, shumëzimi i *saktë* i matricave me numra të plotë 1 015–131 063 bitësh del bit për bit i saktë, 10–19× më shpejt se FLINT dhe me 15–18× më pak energji. Përcaktori i saktë i një matrice int8 del 224× më shpejt se FLINT te $n = 4096$ dhe 355× te $n = 8192$, me një kufi të provuar që bie brenda një biti te këto madhësi; te $n = 32\,768$ (425 348 bit) mbaron për 594–615 s brenda 12 GB të GPU-së. Për hyrje 32–256 bit, ngritja e rendit të lartë mbi të njëjtat GEMM modulare dhe një kufi i provuar për hyrje me plane bajtesh e japin përcaktorin 291× më shpejt se FLINT te $n = 2048$ me hyrje 64-bit, 359× me 128-bit dhe 404× te $n = 4096$; raporti rritet me madhësinë e hyrjeve. Te 15 nga sistemet reale

më të keqkushtëzuara të NIST Matrix Market zgjidhja e saktë e certifikuar del për 0,17–115 s, 9–7 857× më shpejt se SPEX aty ku SPEX mbaron, ndërsa FP64 s'ka asnjë shifër të saktë te 8 prej tyre. Algoritmi i Strassen-it modulo m, i saktë për çdo modul, arrin 91 % të shpejtësisë INT8 që laptopi mban, dhe një DGEMM me saktësi FP64 te $n = 16\,384$ kushton 73,6 J, 13,9× më pak se FP64-ja e vetë GPU-së në pikën e saj më të mirë. Të njëjtat njësi japin prodhime MXFP4, NVFP4, FP8 dhe BF16 të rrumbullakuara saktë mbi peshat reale të modeleve gjuhësore të hapura deri 2,78 mijë miliardë parametra, të njëjta për çdo madhësi grupi; Nemotron 3 Ultra (560 miliardë parametra) xhiron i tëri nga disku i laptopit, pa kosto në dekodim.

1 Introduction

The FP64 units of consumer and laptop GPUs are a small fraction of the chip: on our laptop Blackwell part a peak kernel reaches 0.449 TFLOPS of FP64 against 28.2 TFLOPS of FP32 and 232 TOPS of INT8 tensor-core throughput (Table 1). Ozaki, Uchino and Imamura showed that FP64 matrix multiplication can be emulated on INT8 tensor cores (Ozaki scheme II): each input is scaled to integers, reduced modulo a set of pairwise coprime moduli, the residue matrices are multiplied exactly in INT8 with INT32 accumulation, and the integer product is reconstructed by the Chinese remainder theorem (CRT) before being scaled back [12]. FP64 by this route is not new; it is our baseline.

What we add, made concrete and measured on a GPU, is that the same machinery does not stop at FP64 (on CPU INT8 units Kouya takes Ozaki-II from 53 to 2048 bits [25]). The precision is set by the number of moduli and by how many floating-point words each input and output carries; the expensive part, the INT8 GEMMs, runs at the same speed whatever the precision. This turns precision into a software choice on hardware that has no wide floating-point unit at all. We build and measure:

- FP64 GEMM at 16 moduli: 6.2–6.8 TFLOPS, 14–15× the native FP64 peak, with smaller error than native FP64; on the same laptop cuBLAS 13's own emulation and GEMMul8 are faster and at least as accurate (Section 5), so FP64 is our baseline, not our claim;
- double-double (∼106-bit) GEMM at 32 moduli: 3.81 TFLOPS, 8.5× the native FP64 peak with twice the bits;
- triple-double (∼159-bit) GEMM at 49 moduli—after which every prime below 256 divides one of them, so no further 8-bit modulus can join—at 2.00 TFLOPS;
- arbitrary precision beyond that from balanced 160-bit slices: 212, 318 and 424 bits;
- dense linear solves at condition numbers where FP64 and double-double fail, all on the GPU: 22× (double-double, $n = 8192$) and 9× (triple-double, $n = 4096$) sooner than the same LUs cache-blocked on the laptop's 24 CPU threads;
- exact products of integer matrices with 1 015- to 131 063-bit entries, the GEMM work linear in the bits (10–19× faster than FLINT, Section 7);
- exact determinants of int8 matrices up to $n = 32\,768$ (40–355× faster than FLINT), and of matrices with 32–256-bit entries by high-order lifting on tensor cores (17–404×);
- certified exact solutions of 15 of the most ill-conditioned real systems of NIST Matrix Market in 0.17–115 s, 9–7 857× sooner than SPEX where it finishes (Section 9);
- Strassen's algorithm over $\mathbb{Z}/m$ inside the emulation: an FP64-accurate DGEMM for 73.6 J at $n = 16\,384$, 13.9× fewer joules than the GPU's own FP64 at its best clock (Section 12);

Table 1: Peak throughput of the laptop GPU (`bench/peak.cu`).

Unit	Throughput
FP64 FMA, native	0.449 TFLOPS
FP32 FMA, native	28.2 TFLOPS
INT8 `mma.sync.m16n8k32` (PTX)	232 TOPS
INT8 / FP64	**517**×

- correctly rounded MXFP4, NVFP4, FP8 and BF16 products on the real weights of open models of up to 2.78 trillion parameters, and the whole of Nemotron 3 Ultra run from the laptop's disk that way, batch-invariant by construction (Section 10).

Everything runs on one laptop, and all competitors were measured on the same machine.

2 Method

Ozaki scheme II. For $C = AB$ the rows of A and columns of B are scaled by powers of two to integers A', B' whose product cannot exceed $M/2$ in magnitude, where $M = \prod_\ell m_\ell$ is the product of L pairwise coprime moduli $m_\ell \le 256$. For each ℓ the residues $A' \bmod m_\ell$ and $B' \bmod m_\ell$ fit in INT8, their product is computed exactly by the tensor cores with INT32 accumulation, and reduced mod m_ℓ. The CRT then gives $A'B'$ exactly as a signed integer of up to $\log_2 M$ bits, which is scaled back. The precision of the result is limited only by how many bits of each input survive the scaling, i.e. by $\log_2 M$.

Wider inputs and outputs. For double-double and triple-double inputs, each of the two or three parts is split exactly into 53-bit integer pieces before the residues are formed; like pieces are summed and cut once into 16-bit chunks, so the residue of a wide integer costs 32-bit multiply-adds per modulus instead of 64-bit reductions (triple-double residues 57 → 19 ms at $n = 4096$). On the output side the integer is never converted through floating-point arithmetic: the CRT is evaluated as a direct sum $x = \sum_\ell r_\ell W_\ell - qM$ over 32-bit limbs on the integer units, with q from 40-bit fixed-point fractions, and the result is peeled off the integer as two or three doubles (round down, round down, round to nearest, each exact from the integer), then normalized. This replaced a Garner reconstruction followed by a double-double Horner evaluation, which cost 2.8 ns per output element (47 ms for 4096^2 outputs, 41.5 ms of it in those two steps) because FP64 runs at 1/64 rate on this chip; the direct sum costs 0.68 ns per output element (11 ms) and is nearly independent of K. It was checked on the host against 10^5 random 200-bit integers with no error.

Moduli. FP64 uses 16 moduli, double-double 32, triple-double 49 (the 32 plus the 17 primes $103 \ldots 29$ that are not yet factors; $M \approx 2^{342}$). With these, every prime below 256 divides one of the moduli, so no further 8-bit modulus coprime to all of them exists and M cannot grow within one GEMM. Beyond ∼160-bit inputs we cut each input into balanced 160-bit slices and run one 49-modulus GEMM per slice pair that reaches the result, accumulating exactly in wide integers and peeling Q doubles.

3 Accuracy

Errors are the maximum componentwise $|C - C_{\text{ref}}|/(|A||B|)$ in units of the target precision's unit roundoff, against exact CPU references: for FP64 the products split exactly (TwoProd) and summed as an unrounded double-double (Dot2 without its final rounding, off by at most $\sim (K2^{-53})^2|A||B|$), for double-double and triple-double an exact 600-bit MPFR sum, beyond

that an exact 2400-bit sum. The 80-bit and `__float128` sums we used at first are themselves off by up to 0.012 units of 2^{-53} and 0.41 units of 2^{-106} on these inputs. Inputs have magnitudes spread over $2^{\pm 0}$ or $2^{\pm 8}$.

Table 2: Error in units of 2^{-53} (FP64, 256×256 outputs) and 2^{-106} (double-double, 128×128 outputs).

K	magnitudes	native FP64	14 moduli	**16 moduli**
512	$2^{\pm 0}$	4.71	10.9	**0.99**
512	$2^{\pm 8}$	24.1	395	**3.1**
4096	$2^{\pm 0}$	4.15	10.5	**0.24**
4096	$2^{\pm 8}$	25.0	203	**1.87**
K	magnitudes	24 moduli	28 moduli	**32 moduli**
256	$2^{\pm 0}$	$6.2 \cdot 10^5$	27.3	**0.71**
256	$2^{\pm 8}$	$2.0 \cdot 10^7$	1160	**47.1**
1024	$2^{\pm 0}$	$4.7 \cdot 10^5$	31	**0.41**
1024	$2^{\pm 8}$	$1.1 \cdot 10^7$	713	**12.6**

Table 3: Triple-double error in units of 2^{-159}, 128×128 outputs, against an exact MPFR sum.

K	magnitudes	41 moduli	45 moduli	**49 moduli**
256	$2^{\pm 0}$	$1.7 \cdot 10^4$	5.2	**0.24**
256	$2^{\pm 8}$	$1.2 \cdot 10^6$	343	**10.7**
1024	$2^{\pm 0}$	$1.8 \cdot 10^4$	4.6	**0.13**
1024	$2^{\pm 8}$	$4.3 \cdot 10^5$	110	**2.1**

Beyond triple-double, against an exact 2400-bit sum at $K = 256$: 0.13 ulp at 212 bits, 0.24 at 318 bits, 0.12 at 424 bits (0.75, 17.4 and 0.54 with magnitudes spread over $2^{\pm 8}$).

4 Speed

Table 4: GEMM speed on the laptop with the machine to itself. FP64 and dd: 3 November (`bench/speed_all.sh`); the rows at $n = 2048$: 4 November (`bench/cpu_base.sh`), GPU and CPU in the same held window, each CPU time the median of five full runs at $n = 2048$ (none scaled), the triple-double CPU code built with `-march=native` (without it `std::fma` is a libm call and the loop does not vectorize: 3.54 s instead of 2.46).

bits	n	GPU time	GPU TFLOPS	best CPU code, 24 threads	MPFR, 24 threads
53 (FP64)	4096	20.3 ms	6.76 (15.1× native)	—	—
106 (dd)	4096	36.1 ms	3.81 (8.5× native)	—	—
159 (td)	4096	67.7 ms	2.03	—	—
159 (td)	2048	11.3 ms	1.52	td AVX2 2.46 s (218×)	22.8 s (2018×)
212	2048	34.2 ms	0.50	QD `qd_real` 15.0 s (439×)	26.4 s (772×)
318	2048	44.8 ms	0.38	—	30.6 s (682×)
424	2048	69.4 ms	0.25	—	35.3 s (508×)

At 16 moduli a 4096^3 FP64 GEMM now takes 20.3 ms. In the code of 15 October (24.7 ms, 5.56 TFLOPS) it split into scales 1.0 ms, residues 3.0 ms, the 16 INT8 GEMMs 16.8 ms (131 TOPS) and the CRT 3.6 ms. For double-double the direct-sum CRT of Section 2 (2.8

to 0.68 ns per output, ∼36 ms) accounts for most of the doubling from 1.76 to 3.81 TFLOPS: 36 of the 42 ms saved out of 78. On its own the INT8 GEMM reaches 157 TOPS, 68% of the measured peak, with a three-stage `cp.async` pipeline; with real data the laptop is power-bound (Section 11), so kernel speed and end-to-end time do not move together.

5 FP64 emulation on one laptop

FP64 from INT8 is not our claim, and two public implementations of it run on this laptop: cuBLAS 13's own emulation (`CUBLAS_COMPUTE_64F_EMULATED_FIXEDPOINT`, cuBLAS 13.8.2 from NVIDIA's wheels, 8 GB workspace; its automatic dynamic precision, ADP, or a fixed number of mantissa bits) [15, 16] and GEMMul8 [17] (commit 30b5b5d, INT8 backend, fast and accurate modes). We ran the three in one held window with one harness (`bench/emu_harness.h`, `bench/fair_jobs.sh`, 4 November 2026): the same inputs (normal values times 2^u, u uniform in $[-s, s]$), the error componentwise against an exact-to-2^{-80} reference at $M = N = 256$, and the time as the median of five calls after a warm-up.

Table 5: FP64 by emulation on the INT8 tensor cores of the laptop, three implementations in one window. Error: $\max_{ij} |C - C_{\text{ref}}|_{ij} / (|A||B|)_{ij}$ in units of 2^{-53} at $K = 4096$, exponents spread over $2^{\pm s}$; speed in FP64-equivalent TFLOPS (median of five); energy at $n = 4096$, three 12-s runs each (`bench/energy_emu.sh`, the same inputs for all).

	error at $K = 4096$, $s =$				TFLOPS at $n =$			GFLOPS/W
	0	8	16	32	2048	4096	8192	$n = 4096$
native FP64 (cuBLAS DGEMM)	0.81	6.3	12.9	13.8	0.41	0.44	0.45	7.5–8.1
ours, 14 moduli	10.5	203	800	3 680	5.41	7.05	5.77	60.0–60.8
ours, 16 moduli	0.24	1.9	4.7	31.5	4.88	6.73	5.06	54.0–55.2
cuBLAS, ADP	0.13	0.65	1.3	13	3.16	5.26	7.02	46.9–48.4
cuBLAS, 47 bits	22.2	389	1 170	6 510	6.59	9.23	10.55	75.3–76.8
cuBLAS, 55 bits	0.12	1.5	4.7	31.5	6.12	8.51	9.67	68.4–69.2
GEMMul8 fast, 14 moduli	3.4	20.2	40.5	222	6.70	9.16	10.69	70.6–72.5
GEMMul8 fast, 16 moduli	0.097	0.39	0.64	1.3	5.86	8.29	9.20	64.4–64.6
GEMMul8 accurate, 16 moduli	0.097	0.39	0.64	1.05	4.97	5.90	8.19	56.9–57.1

At FP64 accuracy cuBLAS's 55-bit mode and our 16 moduli are within a factor of two of each other's error at every spread (0.12 and 0.24 units at $s = 0$, 4.7 both at $s = 16$, 31.5 both at $s = 32$), and cuBLAS is 1.26× faster at $n = 4096$ (8.51 against 6.73 TFLOPS) and 1.91× at $n = 8192$, where ours falls back to 5.06 TFLOPS while cuBLAS keeps rising (we have not profiled the fall). GEMMul8 at 16 moduli is the most accurate of the three at every spread (0.097–1.3 units in its fast mode, 1.05 at $s = 32$ in its accurate mode) and runs at 8.29 TFLOPS at $n = 4096$ in fast mode. Per joule, cuBLAS's 55 bits do 26% more work than our 16 moduli and GEMMul8's fast mode 18% more. NVIDIA reports up to 13× over native FP64 on the RTX PRO 6000 Blackwell Server Edition [15]; on this laptop, whose native DGEMM runs at 0.41–0.45 TFLOPS, the 55-bit emulation is 15–22× it. For FP64 itself, then, use theirs. What this paper adds starts past FP64: the same INT8 path at 106 to 424 bits (Section 4) and the solves below, where FP64 has no correct digit.

6 A real task: ill-conditioned linear systems

We solve $Ax = b$ with $A = Q_1 \operatorname{diag}(\sigma) Q_2$, $Q_{1,2}$ orthogonal (a DST-I, the second permuted and sign-flipped), σ spaced from 1 to 10^{-20} or 10^{-36}; such matrices cannot be fixed by scaling. $b = Ax_{\text{true}}$ is summed exactly. The error is $\max |x - x_{\text{ref}}| / \max |x_{\text{ref}}|$.

Double-double, $\kappa = 10^{20}$. κu is 10^4 for FP64, so FP64 LU has no correct digit and textbook iterative refinement diverges (it needs $\kappa u < 1$; at $\kappa = 10^{12}$ the same refinement converges to $3 \cdot 10^{-20}$, so the baseline is right). The GPU solver keeps everything on the device: a recursive panel with cooperative 32-column leaves (a thread per row holding the row in registers, rows renamed instead of swapped, one grid synchronization per column), halves joined by double-double GEMMs on the tensor cores, L_{11}^{-1} assembled from the halves' inverses, left-looking solves with inverted diagonal blocks, and one refinement step in double-double.

Table 6: Double-double LU, $\kappa = 10^{20}$, one run per n (seed 1, `bench/dd_same_run.sh`, 4 November 2026): the GPU solver (nb = 1024) beside the same LU on 24 CPU threads (AVX2; cache-blocked with nb = 64, and unblocked: bit for bit the same result) and a `__float128` LU on 24 threads. GPU speedups in parentheses. Error vs x_{true} in units of 10^{-11}.

		dd LU, 24 CPU threads			error vs x_{true}
n	**dd LU, GPU**	blocked	unblocked	`__float128` LU	GPU / CPU / quad
2048	**0.083 s**	0.30 s (3.6×)	0.42 s (5.1×)	8.91 s (107×)	2.8 / 0.6 / 2.4
4096	**0.223 s**	1.91 s (8.6×)	9.24 s (41×)	56.2 s (252×)	1.4 / 3.1 / 1.8
6144	**0.419 s**	6.15 s (15×)	38.9 s (93×)	185.6 s (443×)	24 / 25 / 29
8192	**0.692 s**	15.52 s (**22**×)	99.5 s (144×)	435.4 s (**629**×)	2.6 / 5.4 / 2.3

Each row is one process: the GPU solve after a warm-up, then the three CPU LUs. Twenty back-to-back GPU solves (`bench/fair_jobs.sh`) give medians of 0.080, 0.210, 0.417 and 0.726 s at the same n, within 6% of the single runs. Blocking makes the CPU LU 1.4–6.4× faster (it is memory-bound otherwise), and the blocked LU is the baseline we quote: against it the GPU's lead grows with n, from 3.6× to 22×, because the work moves into the tensor-core GEMMs; against `__float128`, the quad precision a user would reach for, it is 107–629×. The errors of all three are of one order, 5–240 times $\kappa u \approx 1.2 \cdot 10^{-12}$ with $u = 2^{-106}$: the conditioning sets them, not the solver.

Triple-double, $\kappa = 10^{36}$. Here κu is 10^{20} for FP64 and 10^4 for double-double: neither has a correct digit. The same solver structure in three parts per number, with 16-column leaves:

Table 7: Triple-double LU, $\kappa = 10^{36}$, one run per n (seed 1, `bench/td_same_run.sh`, 4 November 2026): time and error vs x_{true}; GPU speedups in parentheses. The CPU LU in the same arithmetic on 24 threads (AVX2), cache-blocked with nb = 64 and unblocked, gives bit for bit the same factors and solution. Errors in units of 10^{-14}; FP64 LU: its error only.

		td LU, 24 CPU threads			error vs x_{true}	
n	**td LU, GPU**	blocked	unblocked	MPFR LU, 160 bits	GPU / CPU / MPFR	FP64 LU
1024	**0.073 s**	0.10 s (1.4×)	0.08 s (1.1×)	1.51 s (21×)	1.3 / 3.6 / 3.3	128
2048	**0.171 s**	0.59 s (3.5×)	0.99 s (5.8×)	11.1 s (65×)	2.1 / 7.6 / 5.6	103
4096	**0.447 s**	3.83 s (**8.6**×)	17.5 s (39×)	74.1 s (**166**×)	3.5 / 12 / 9.6	86

At $n \leq 1024$ a well-vectorized triple-double LU on 24 cores keeps up; the gap opens with n as the work moves into the tensor-core GEMMs. At $n = 4096$ the panels take 0.38 s of the 0.563 s (leaves 0.14 s): the panel, not the GEMM, is now the limit.

7 Thousands of bits, exactly: linear in the bits

The 49 moduli ≤ 256 cap one Ozaki-II product at ∼160-bit inputs, and slicing beyond that costs a number of GEMMs that grows with the square of the slices. For wide integers we leave 8-bit moduli: the moduli are primes $p < 65\,280$, and each balanced residue ($|r| < p/2$) is split into

two signed bytes, $r = 256h + \ell$. Then $AB \bmod p$ is four exact INT8 GEMMs (hh, $h\ell$, ℓh, $\ell\ell$) recombined in 64-bit integers, so *the GEMM work grows linearly with the number of bits.* Above ∼60 000 bits of the CRT modulus the residues take three bytes and the primes run up to 2^{23} (nine products per prime, ∼23 bits each). The two conversions are INT8 GEMMs as well, as conversions to and from a residue number system are matrix products: the residues are the balanced byte digits of each entry times the bytes of $2^{8j} \bmod p$; for the CRT, $t_\ell = r_\ell\,(M/p_\ell)^{-1} \bmod p_\ell$ (bytes) times the bytes of M/p_ℓ gives the digits of $x+qM$, with $q = \mathrm{round}(\sum_\ell t_\ell/p_\ell)$ from doubles, and qM is taken off while the digits are carried. Everything runs on the tensor cores (`src/ozaki_big.cuh`, `bench/big_test.cu`).

Every product is exact: 256 sampled elements per run agree bit for bit with GMP. The baselines compute the same exact product on all 24 CPU threads: GMP inner products (`mpz_addmul`), and FLINT's `fmpz_mat_mul`, which is itself multimodular with BLAS-style kernels and is the strongest baseline we know of.

entry bits	n	primes (bytes)	GPU	GMP	FLINT	GPU vs FLINT
1 015	1024	192 (2)	71 ms	23.0 s	1.31 s	18×
4 087	512	576 (2)	62 ms	16.4 s	1.07 s	17×
16 375	256	2 112 (2)	90 ms	14.0 s	1.69 s	19×
4 087	1024	640 (2)	341 ms	125 s	6.10 s	18×
16 375	512	2 112 (2)	363 ms	104 s	6.33 s	17×
32 759	256	4 352 (2)	316 ms	36.1 s	4.56 s	14×
65 527	128	5 760 (3)	313 ms	10.6 s	3.45 s	11×
131 063	128	11 520 (3)	983 ms	29.6 s	9.76 s	10×

GMP times are measured on a corner of the outputs (96^2, or 48^2 above 8 000 bits) and scaled; FLINT runs the full product. The largest entries have ∼39 450 decimal digits. Energy per exact product (RAPL package + `nvidia-smi`, both chips counted in both columns): 1 015 bits, $n = 1024$: 8.6 J on the GPU path against 131 J for FLINT (15×); 16 375 bits, $n = 256$: 7.9 J against 142 J (18×).

Doubling n at fixed width does not raise the ratio either (16 375 bits: 19× at $n = 256$, 17× at $n = 512$): even at $n = 512$ the conversion GEMMs ($\sim 1.9 \cdot 10^{13}$ INT8 operations) dwarf the 8 448 product GEMMs (2 112 primes, four each) ($\sim 2.3 \cdot 10^{12}$). The ratio to FLINT falls slowly beyond ∼16 000 bits (19× → 10×) because the conversions are n^2P^2 work for P-bit entries while the products are n^3P: at $n = 128$ the conversions dominate. The way past that crossover is a subquadratic conversion (a two-level residue system), which we leave for the next version. For scale: one part in a googol (10^{-100}) is ∼333 bits, already inside the 424-bit format of the previous section.

8 Changing the problem: exact determinants and exact solutions

The ratio of the previous section falls with the bits because large numbers must enter and leave the residue system. Many exact computations produce huge numbers from small inputs: the determinant of an $n \times n$ int8 matrix has ∼0.72 bits per row less than Hadamard's bound, about 47 000 bits at $n = 4096$, and the rational solution of $Ax = b$ has n numerators of that size. There the conversions vanish: the input is int8, each prime returns one residue (or a vector of digits), and the only big-number step is on the host (`src/ozaki_det.cuh`, `bench/det.cu`).

LU modulo thousands of primes. For primes $p < 65\,280$ and residues X, W, with $X = X_\ell + 256X_h$ in balanced bytes and $V = 256W \bmod p$, $XW \equiv [X_\ell\, X_h][W_\ell; V_\ell] + 256\,[X_\ell\, X_h][W_h; V_h] \pmod p$: two INT8 GEMMs of depth $2K$, reduced once in the epilogue. A recursive LU (Toledo [8]) without pivoting runs for a batch of primes at once, every $O(n^3)$ step such a product. Its 64-column leaves invert their 64×64 diagonal blocks in shared memory. A prime with

a vanishing leading minor (probability $\sim n/p$) is dropped. Structured matrices, whose leading minors vanish over $\mathbb{Z}$, are preconditioned as UAL with random unit triangular U, L (Kaltofen–Saunders [9]) whose entries in $[0, 127]$ are the same residues for every prime.

A divisor by p-adic lifting. (The divisor-then-primes hybrid is classical on CPUs [10, 11, 29]; FLINT uses it.) $A^{-1} \bmod p$ comes from the LU of $[A\ I]$. Dixon lifting [7] runs on the GPU, one 16-bit and one int8 matrix–vector product per step: $x_i = A^{-1} r_i \bmod p$, $r_{i+1} = (r_i - A x_i)/p$. Rational reconstruction of a few entries of $x = A^{-1} b$ gives d, which divides $\det A$ and for random b is almost always the largest invariant factor. Then $\det A/d$ needs only enough primes to exceed a bound on $|\det A|/d$.

A proved bound, within one bit at $n \le 8192$. Hadamard's bound leaves $\sim 0.72n$ bits of slack on a random matrix (5 900 at $n = 8192$), i.e. hundreds of primes for $\det A/d$. We replace it by a certificate in the spirit of Ogita's verified determinants [28]. FP32 LU with partial pivoting gives $PA \approx LU$. $X_L = \lfloor 2^s L^{-1} \rceil$ (unit diagonal, so exactly 2^s) and $X_U = \lfloor 2^s U^{-1} \operatorname{diag}(u) \rceil$ are integer triangular matrices, so their determinants are products of their diagonals. $B = X_L\, PA\, X_U$ is computed *exactly* (the same modular INT8 GEMMs, int64 operands reduced while packing, Garner to 128 bits), and Hadamard's inequality on B, accumulated with upward rounding, bounds $|\det A| = |\det B|/(2^{sn}\,|\det X_U|)$. Floating point only decides how close B is to $2^{2s}\operatorname{diag}(u)$, that is how tight the bound is, never whether it holds. Measured: 47 021 bits against $|\det A|$ of 47 021 bits at $n = 4096$, 98 140 against 98 139 at $n = 8192$. $\det A/d$ then needs 6 primes instead of 224 and 496.

Results. Random int8 matrices, FLINT 3.0.1 on all 24 CPU threads; every result bit-exact against FLINT. FLINT's `fmpz_mat_det` is already its accelerated path (a Dixon divisor, then primes). Its plain multimodular `fmpz_mat_det_modular` takes 347 s at $n = 2048$, 145–177× our all-primes LU with the same algorithm.

n	$\lvert\det A\rvert$ bits	GPU, all primes	GPU, lifting + bound	FLINT	vs FLINT
1 024	10 736	0.25 s	0.073 s	2.89 s	40×
2 048	22 485	1.96–2.4 s	0.293 s	24.9 s	85×
4 096	47 021	21.3 s	1.15 s	257 s	224×
8 192	98 139	—	8.05 s	2 861 s	355×
16 384	204 480	—	65.9 s	—	
32 768	425 348	—	594 s	—	

GPU times are the fastest logged run; repeats of a configuration took up to 22% longer (all primes at $n = 2048$: 1.96–2.4 s; lifting: 0.293–0.321 s at $n = 2048$, 1.15–1.19 s at $n = 4096$), the GPU being power-bound. At $n = 8192$ FLINT ran while some GPU runs shared the CPU; the leading digits agree, and $n = 8192, 16\,384, 32\,768$ are also checked modulo four primes used nowhere else. The exact rational solution of $Ax = b$ (same lifting; all n entries back to integers by Horner in halves on 24 threads; one common denominator) is bit-exact against FLINT's `fmpz_mat_solve_dixon_den`: 0.064 s against 2.33 s at $n = 1024$ (36×), 0.145 s against 18.9 s at $n = 2048$ (130×), 1.62 s against 145 s at $n = 4096$ (90×, 192.6 Mbit of numerators), and 7.5 s at $n = 8192$ (804 Mbit). Energy per determinant at $n = 2048$, both chips counted: ∼40 J on the GPU path (process start-up included; ∼23 J for the computation alone) against ∼2 030 J for FLINT, ∼50× less. At $n = 8192$ the lifting is the cost: 13 013 steps, each reading A^{-1} (16 bits per entry) and A, at ∼100% of the read bandwidth we measure on this GPU (∼410 GB/s). That limit is physical for digit-by-digit lifting; moving past it needs lifting built on matrix products (high-order lifting, cf. the DoublePlusOneLift of Pauderis and Storjohann [31]), which the paragraphs below build for entries of 32 bits and more; for int8 entries at these sizes it is the next step.

$n = 32\,768$ **in 12 GB.** One modular product packs $(2n + 128) \cdot 2n$ bytes three times (13 GB), and the bound holds three float and two int64 $n \times n$ matrices (30 GB). Products deeper than 2048 now run in K-chunks (a third epilogue mode, $R \mathrel{+}= XW$); the bound builds X_L, X_U in column panels (triangular solves against identity panels) kept in host memory, forms $T = X_L\, PA \bmod q$ in row panels and $B = TX_U$ in column panels as 16-bit residues on the host, and sums the rows in panels, the L_1 norms exactly in 128 bits; it runs on its own stream beside the lifting. Peak use: ∼11 GB of VRAM and 29 GB of host memory. The determinant (425 348 bits) takes 594–615 s in three runs (the fastest: A^{-1} 15 s, 56 145 lifting steps 497 s at the bandwidth limit, $\det A/d$ 39 primes 71 s); the bound, 242–244 bits above $|\det A|$ at this size (FP32 LU; a larger scale 2^s does not change it), is hidden under the lifting. Each piece was checked bit-exact against FLINT at $n \le 4096$ with the chunks and panels forced small.

Large entries. With entries of L bytes stored as byte planes ($A = \sum_j 256^j A_j$), the residues come by Horner and the LU is unchanged; the lifting reads the L planes per step and keeps the residual as $L + 4$ base-256 digits per row, divided by p exactly by one lane. Lifting modulo $M = p_0 \cdots p_{S-1}$ ($S \approx L/2$ primes at once: x_i by CRT from S residues, Ax_i on byte-aligned digits, division by each p_t in turn) gains $16S$ bits per read of the L planes and cuts the lifting 1.3–2.9×. Bit-exact against FLINT: $n = 1024$ with 64-bit entries in 0.247 s against 12.4 s (50×), with 128-bit entries 0.692 s against 31.0 s (45×); $n = 2048$ with 32-bit entries 0.563 s against 46.7 s (83×), with 64-bit entries (a 137 173-bit determinant) 2.03 s against 106.9 s (53×). These runs predate the byte-plane bound and the half-gcd reconstruction described below, which is why the Dixon column of Table 8 is faster. GPU times come from runs without FLINT beside them: CPU and GPU share one power budget, and right after 24 FLINT threads the GPU is up to 2× slower. Each step still reads all L planes, so the cost grows faster than the bits.

High-order lifting. Digit-by-digit lifting reads A and A^{-1} once per X-digit; high-order lifting (Storjohann's sparse inverse expansion [30], in row form $yA = b$) squares the residue instead, so the precision doubles per level. With $X = p_0 \cdots p_{s-1} \ge 6.25\, n^2 \|A\|$ (distinct 16-bit primes, $B_0 = A^{-1} \bmod X$ from the same batched LU), $R_0 = (I - AB_0)/X$ and, per level, $\bar{R} = R_i^2$, $M_i = B_0 \bar{R} \bmod X$, $R_{i+1} = (\bar{R} - AM_i)/X$ exactly, with $\|R_i\| \le 1.25\, n \|A\|$; then $c_{i+1} = c_i + X^{m_i} c_i R_i + X^{2m_i} b M_i$, $m_{i+1} = 2m_i + 1$, gives $c_i \equiv bA^{-1} \pmod{X^{m_i}}$. The residue chain runs in RNS, modulo the s primes of X and w more primes ($\prod q > 2.5\, n \|A\|$): M_i by Garner from its residues, $R_{i+1} \bmod q = (\bar{R} - AM_i)X^{-1}$, its exact value by Garner over the q (checked against its bound) — $2(s + w)$ modular GEMMs per level and no big number anywhere. All matrices are kept transposed so that the vector chain reads K-contiguous rows. That chain is itself a GEMM: c_i is stored as balanced digits in a periodic mixed radix (position e has radix $p_{e \bmod s}$), so X^m is a shift by sm positions and the Garner digits of M_i are already digits of it; rows are the digits as [lo | hi] bytes, columns the byte planes of R_i as $[Z_v \,|\, Z_{v-1}]$, so column v collects 256^v; the planes are combined in parallel and the carries run down the positions one thread per entry. Around it, the proved bound now takes byte planes (a float LU of $A/256^{L-1}$ only decides tightness; $B = X_L\, PA\, X_U$ is exact from the planes, Garner over up to 16 primes with $|B_{ij}|$ rounded up from the mixed-radix digits): at $n = 2048$ with 64-bit entries it is 1 bit above $|\det A|$, and $\det A/d$ needs 6 primes instead of 128. The rational reconstruction is FLINT's half-gcd `_fmpq_reconstruct_fmpz_2` (0.49 s → 0.047 s there). Bit-exact against FLINT in every case of Table 8: the lifting step is 1.4–4.3× faster than S-prime Dixon's (at $n = 2048$: 1.4, 3.0 and 4.3× for 32-, 64- and 128-bit entries; whole determinants, as in the table, 1.2–3.5×) and grows like the bits, not their square, so the ratio to FLINT now rises with the entry size—63× at $n = 512$ with 256-bit entries, 291× at $n = 2048$ with 64-bit and 359× with 128-bit entries, 404× at $n = 4096$ with 64-bit entries (a 276 398-bit determinant in 2.15 s against 868 s, with the residue chain over 8-bit moduli below). In a 3.27 s run with 16-bit primes, the residue chain was 2.08 s of the 2.71 s lifting at that size (247 modular GEMMs of n^3, power-bound), the vector chain 0.44 s. Moduli below 256 for X (one GEMM per modulus, as in Ozaki-II) are not an option

Table 8: Exact determinants of $n \times n$ matrices with random b-bit entries: total GPU time with S-prime Dixon lifting and with high-order lifting, its residue chain over 16-bit primes (HOL) or over 8-bit moduli (HOL_8), all with the proved bound for byte planes and FLINT's reconstruction, against FLINT `fmpz_mat_det` on 24 threads. GPU runs without FLINT beside them, the fastest logged; all bit-exact (HOL_8: the same determinants as HOL, `bench/hol8_time.sh`, machine held). —: not measured, or not applicable (entries above 128 bits fall back to 16-bit primes).

n	b	$\lvert\det A\rvert$ bits	Dixon	HOL	HOL_8	FLINT	vs FLINT
256	256	65 914	0.139 s	0.063 s	—	1.08 s	17×
512	256	132 082	0.399 s	0.139 s	—	8.82 s	63×
1024	64	68 073	0.177 s	0.126 s	0.119 s	12.39 s	104×
1024	128	133 615	0.434 s	0.219 s	0.196 s	30.98 s	158×
2048	32	71 638	0.358 s	0.288 s	—	46.70 s	162×
2048	64	137 173	1.091 s	0.453 s	0.367 s	106.89 s	291×
2048	128	268 249	2.946 s	0.843 s	0.728 s	261.65 s	359×
4096	32	145 330	3.425 s	1.934 s	—	416.46 s	215×
4096	64	276 398	7.443 s	3.004 s	2.147 s	867.68 s	404×

here: without pivoting the LU modulo p fails with probability $\approx 1 - e^{-n/p}$. They are an option for the residue chain wherever B_0 is not involved: $\bar{R} = R_i^2$ exactly in an RNS of 23–42 odd moduli below 256 (one INT8 GEMM each; on random data a 16-bit modular product costs 2.8× more per bit of modulus at $n = 4096$), a base extension by approximate CRT to the 16-bit primes of X, and AM_i modulo the first 10–19 moduli only. Measured in one session with the machine held, it takes 4096×64 bits from 3.00 to 2.15 s (404× FLINT) and 2048×128 bits from 0.84 to 0.73 s, with the same determinant as the 16-bit chain in every case (Table 8).

9 Real systems: how wrong FP64 is, and the exact answer

We took the 13 most ill-conditioned square real matrices with $900 \le n \le 8192$ of NIST Matrix Market, and two larger ones (fidapm15, cry10000; solved by Dixon lifting)—fluid dynamics (FIDAP, LNS), magnetohydrodynamics (MHD), the Quebec Hydro network, crystal growth, oil reservoirs, a chemical plant—and solved $Ax = b$ *exactly*. Every double is $m\,2^e$, so scaling row i by 2^{k_i} gives integer entries (up to 154 bits) and the same x, with no rounding anywhere: an integer system $A'x = b'$. Its rows are ordered by FP64 partial pivoting, since the GPU LU does not pivot. High-order lifting (in row form, on A'^{T}) gives all n entries, a common denominator follows from a few half-gcd reconstructions, the reconstruction is unique by the Hadamard and Cramer bounds, and $A'x = b'$ is checked modulo two further primes. On the same integers we ran SPEX Left LU [4] (SuiteSparse 7.6, exact sparse REF LU, one CPU thread); both programs print a digest $\sum_j x_j r^j \bmod 2^{61} - 1$. FP64 is LAPACK `dgesv` and SuperLU on the original doubles.

On 7 of the 13 systems (8 of all 15) FP64 has not one correct digit, LAPACK and SuperLU alike—on mhd3200a and mhd4800a the normwise error is 10^8 and 82–99 % of the entries have no correct digit—and NIST's own (FP64) condition estimate does not say which: qh1484 at $5.6 \cdot 10^{17}$ keeps 11 digits. The certified exact solution takes 0.17–34.4 s (78.7 and 115.0 s for the two larger systems); SPEX returns the same x, bit for bit, where it finishes within the limit, 9 to 7 857× later.

matrix	n	κ (NIST)	bits	GPU s	SPEX s		same x	FP64 digits (entries with none)	
								LAPACK	SuperLU
west0989	989	$5.7{\cdot}10^{12}$	75	0.17	1.5	9×	yes	11.7 (0)	11.8 (0)
qh1484	1484	$5.6{\cdot}10^{17}$	96	0.55	12.4	23×	yes	11.2 (0)	12.2 (0)
fidap032	1159	$5.2{\cdot}10^{20}$	60	0.28	54.5	193×	yes	0.0 (568)	0.0 (572)
fidap006	1651	$3.5{\cdot}10^{21}$	154	1.08	963.7	895×	yes	0.0 (887)	0.0 (693)
cry2500	2500	$4.4{\cdot}10^{17}$	70	1.41	699.3	497×	yes	6.4 (0)	5.1 (0)
fidapm07	2065	$6.8{\cdot}10^{19}$	118	1.19	9 381.8[c]	7 857×	yes	0.0 (430)	0.0 (290)
fidap026	2163	$4.7{\cdot}10^{18}$	153	2.12	9 833.9[c]	4 643×	yes	0.0 (1568)	0.0 (1530)
mhd3200a	3200	$6.5{\cdot}10^{39}$	130	4.02	1290.8	321×	yes	0.0 (2876)	0.0 (2768)
fidap014	3251	$2.6{\cdot}10^{16}$	124	4.74	>7200		–	7.6 (0)	8.0 (0)
lns_3937	3937	$1.0{\cdot}10^{17}$	79	4.31	5 475.3[a]	1 272×	yes	10.1 (2)	7.4 (1)
fidap012	3973	$2.2{\cdot}10^{20}$	119	7.44	10 422.9[b]	1 400×	yes	0.0 (483)	0.0 (482)
mhd4800a	4800	$3.2{\cdot}10^{47}$	131	34.39	4 341.4[b]	126×	yes	0.0 (3923)	0.0 (4754)
sherman3	5005	$6.9{\cdot}10^{16}$	102	14.25	13 517.7[c]	949×	yes	11.8 (0)	12.4 (0)
fidapm15	9287	$1.36{\cdot}10^{20}$	63	78.66	–		–	0.0 (2251)	0.0 (5260)
cry10000	10000	$6.9{\cdot}10^{20}$	76	115.02	–		–	4.2 (66)	1.6 (923)

Table 9: Exact solutions of real systems on the laptop (GPU: A^{-1}, lifting, reconstruction and all numerators). SPEX ran afterwards on the CPU alone, limit 2 h; [a]3 h limit, beside a 290 GB download; [b]3 h limit, at night after the GPU jobs; [c]8 h limit, the machine otherwise idle (sherman3: 14 141.6 and 14 369.0 s on two earlier nights beside light load; fidap026: low-priority compilations, one or two jobs, ran during its last 19 minutes); fidap014: 8 h run pending at the time of writing; –: not run. Where SPEX finished, its digest equals the GPU's. $A'x = b'$ held modulo both check primes in every equation.

10 The largest open model: correctly rounded MXFP4 and BF16 products

Kimi K3 (Moonshot AI; read from its Hugging Face model card, `config.json` and safetensors headers: 2 779 931 837 184 parameters, 1.56 TB) is, to our knowledge, the largest open-weights model (October 2026). Its routed experts (2.72 T parameters; 92 MoE layers of its 93, the first being dense, × 896 experts, 16 per token) are stored in MXFP4: an E2M1 value per weight, one E8M0 scale per 32. A group of 32 is the depth of one `mma.m16n8k32` INT8 step and 2·E2M1 is an integer ≤ 12, so MXFP4×BF16 products can be computed *exactly* on INT8 tensor cores and rounded once (`llm/mxexact.cu`, `llm/exact.cuh`). Weights: $q' = 2\,\mathrm{E2M1} \cdot 2^{(s-s_{\min}) \bmod 4}$ is an int8 (≤ 96), one scale per row and band of four binades (every row of the 48 K3 matrices tested spans ≤ 3 binades: one band). Activations: each token is aligned to its largest exponent and cut into S int8 digits ($D_c = 7S - 8$ bits below the top), smaller elements form further tiers. Every (band, tier, digit) pair is a plain INT8 GEMM whose int32 sums cannot overflow ($|P| \leq K \cdot 96 \cdot 128 < 2^{26.4}$); the few terms of an output are added in a 128-bit integer and rounded once, with a certificate when a term falls below the 94-bit window (it never failed). BF16×BF16 is the same with both sides in digits. A correctly rounded result cannot depend on the summation order, the tiling, the batch or the machine.

On the real weights of layer 5 (experts 0–15, all three matrices, fetched by HTTP range requests; Student-t activations scaled by the layer's real norm weight), against what inference engines do (BF16 tensor cores, FP32 accumulation, split-K chosen by shape):

certified correctly rounded (48 matrices, 79.7 M outputs)	all
independent check (Python integers, raw bytes)	1500/1500
batch 1, 3, 17, 64, 200 vs 4096; decode kernel vs prefill	bit-identical
engine outputs not correctly rounded, split-K 1/2/4/8 (expert 0 w1)	81.5/67.1/55.6/51.2%
the same, split-K 1, each of the 48 matrices	75–83%
engine, split-K 1 vs 4: outputs whose bits differ	71%
prefill, 4096 tokens, 3072×3584: exact / BF16	8.0 / 2.9 ms
decode, 1 token: exact (packed 4-bit) / FP4 GEMV, FP32	0.062 / 0.125 ms

On the real router of the layer (896×7168 BF16, FP32 logits, sigmoid plus learned bias, top 16), the rounding of the logits alone sends ∼1 token in 13 000 to a different expert (11 in 147 456 token–split pairs), i.e. over K3's 92 MoE layers ∼1 token in 146 runs a different expert somewhere; with exact logits the choice depends on the input alone. A whole MoE block of layer 5 (62 real experts fetched on demand, the latent projections, RMSNorm and the shared experts; products correctly rounded, reductions in FP64 in a fixed order) gives the same bits for every token alone or in a batch, while the engine path changes 49% of the block's 28 672 BF16 outputs between split-K 1 and 4.

Where exactness costs nothing: a trillion-parameter model on this laptop is a streaming problem. Per text token K3 reads 25.8 GB of routed MXFP4 experts and 111.2 GB of dense BF16 weights (attention 72.4, shared experts 24.3, gates, latent projections and norms 12.1, lm_head 2.4; summed from the 96 safetensors headers); we measured 5.1 GB/s from the NVMe drive and 45 GB/s over PCIe 5 ×16, while the exact decode kernel consumes 94 GB/s. A bit-reproducible run therefore takes the same ∼27 s per token as an ordinary one, all of it reading. What this laptop lacks is 1.56 TB of storage (669 GB free).

The largest model with a published data recipe. Nemotron 3 Ultra [32] publishes weights and recipe and names its pre-training data (560 524 578 816 parameters in the BF16 checkpoint's metadata; 14.8T tokens in 226 datasets), but the bulk of that data (Nemotron-CC v2 and v2.1, the code and SFT pre-training sets) is gated by manual approval under an agreement that allows internal training only (Hugging Face API, 27 October 2026). Its routed experts are NVFP4 for weights and activations: E2M1 values, one E4M3 scale per 16, an FP32 global scale. An NVFP4 block of 16 is exactly one `mma.m16n8k16` INT8 ($|p_b| \le 2304$), the product of the two E4M3 scales is an integer ≤ 225 times $2^{\varepsilon_W + \varepsilon_A}$, and after a shift of 18 every term is below 2^{47}: the $K/16 \le 320$ terms add in an `int64` without loss, and the 104-bit product with $g_W g_A$ is rounded once. On expert 0 of MoE layers 49 and 99 (real weights, 4096 tokens through the real latent projection, up, relu^2, down) the tensor-core path, an integer-loop path, 32 tokens alone, Python rationals from the raw bytes (sampled) and a CPU recomputation of all 29 360 128 outputs of layer 49 (BLAS block sums, Python integers) agree in every bit. The hardware's own NVFP4 instruction, `mma.m16n8k64.kind::mxf4nvf4.block_scale.scale_vec::4X` with FP32 accumulation (what inference engines run), leaves 18–49% of the outputs not correctly rounded and changes up to 0.81% of them between split-K 1 and 4; an emulation with an exact epilogue shows that the FP32 sum alone accounts for 0–0.9% and the rest is $\alpha = \mathrm{fp32}(g_W g_A)$. The native instruction runs at 49–55 TOPS with a plain tiling and the exact path at 20.5–23.2 TOPS with shared-memory tiles (25.6 without its `int64` epilogue: the epilogue costs ∼20%, the rest is four `k16` INT8 MMAs per `k64` FP4 one); sustained for 10 s each at ∼94 W they give 447–473 and 194–204 GOP per joule. The ceiling is the instruction peak: measured in registers, `m16n8k16` INT8 reaches 116 TOPS and the block-scaled FP4 instruction 465 TOPS, so with both kernels tuned to peak a correctly rounded prefill costs 4×; in decode both read the same packed bytes. The exact decode runs at 405–487 GB/s of packed weights for one token and 136–184 GB/s for eight (one matrix 100 times in a row, so partly from the L2 cache: above the ∼410 GB/s we measure from DRAM); the whole NVFP4 checkpoint (352 GB, from its 113 safetensors headers) fits this laptop's free disk, and a token needs at most 51.2 GB of it by the headers (37.8 dense, 13.4 of routed experts; measured

below: 48 GB read and 19.5–19.7 s per token from NVMe). Its FP32 router is exactly three BF16 matrices, so its logits are one correctly rounded product with $K = 3 \cdot 8192$: FP32 accumulation rounds 90–97% of them differently, yet only 1 of 16 384 token-layer routing decisions moves (2 layers, 2 seeds, 4096 tokens).

The whole model, token by token, from disk. One GEMV covers the four formats of the checkpoint: BF16 (attention, latent projections, lm_head), FP32 (router), FP8 E4M3 W8A8 with per-tensor scales and a static input scale (Mamba in and out, shared experts) and NVFP4 W4A4 (routed experts). Once an activation is quantized as its format prescribes, every element is an integer times a power of two, so a warp accumulates its row in a 128-bit fixed point anchored at the row's largest possible term, multiplies by the exact product of the FP32 global scales and rounds once to BF16 (FP32 for router and logits); terms below the 125-bit window are counted and the rounding is certified when the interval they span rounds to one value. Against a 2560-bit superaccumulator on the host, 6 656 outputs in all four formats (narrow and $2^{\pm 60}$ ranges, heavy cancellation, exact ties) agree in every bit; a 40-bit window and ties-away rounding are both caught (the tie test also caught a bug: squeezing the accumulator to 78 bits widened the interval when no bit was lost, so every exact tie came out uncertified). Everything else (the Mamba-2 recurrence with its FP32 state, the causal convolution, attention without positional encoding, sigmoid top-22 routing, RMSNorms, residuals) runs on the host in FP64 and is rounded once to the precision the model keeps there, identically for every path. All 560 B parameters stream from the laptop's NVMe: one sequence reads 48 GB per token and takes 33–37 s, or 19.5–19.7 s when a thread reads the next tensors into a 3 GB pinned ring while the GPU works (same digest); the FP32 engine path takes 20.3–20.9 s the same way, so exactness costs nothing at decode: both wait on the disk. "The capital of Kosovo is" continues " Pristina". Three prompts (Albanian, English, $123456789 \times 987654321$) then ran as nine sequences, each prompt exact and on the same host code with FP32-accumulating GEMVs (split-K 1 or 4: two configurations of one engine), 45 steps of 57–71 s (70–86 GB read): 1.54 billion correctly rounded outputs, none uncertified. On the exact path's own inputs the engine rounds 40.6–47.3% of the FP32 router logits differently but only 0.003% of the FP8 and NVFP4 outputs (41 914 of 1.42 billion); a separate count (one sequence, seven steps) gives 0.01% of the BF16 projections (561 of 4.86 million) and 73–83% of the FP32 output logits. What is rounded to BF16 is almost always exact under FP32 accumulation; what stays in FP32 is not, and the router's logits set the expert weights. Downstream, the engine's top logit drifts up to 1.22 from the exact one (0.17–0.31 on average) while the tokens still agree, the exact top-two margin falls to 0.017, and the streams part: the Albanian one at the 5th generated token with split-K 4 (an indentation of 8 against 4 spaces) and at the 13th with split-K 1 (" ri" against " pav"; the exact path goes on "Kosova është vend i ri i pavarur", the engine "Kosova është vend i pavarur që ka deklaruar pavarësinë më 17 shkurt 2008"), the English one at the 29th (" and" against " of"). On the product all three write 121932631112635269, which is correct. The drivers print a 64-bit FNV-1a digest of every residual stream and logit vector of the exact path, so anyone with the checkpoint can check a run.

Full access. Checking an output against the data a model learned from needs the data, not only its recipe. Measured on the Hugging Face API and the model cards (27 October 2026): Ultra's main sets are gated as above; Jais-3-375B-A23B (379 B, Apache-2.0) beats Ultra on most rows of its own card but publishes a 10% sample of its data (23.8 B tokens); the strongest model whose data is wholly public and ungated is IFM's K2-Think-V2 (72.6 B, Apache-2.0, TxT360 under ODC-BY); and the one model whose entire training data is indexed for public lookup is OLMo-2-0325-32B-Instruct (all 4.6 T tokens in the infini-gram index `v4_olmo-2-0325-32b-instruct_llama`). No fully open model was better than Ultra on that date. The same exact GEMV drives both dense models (`llm/dense.cu`: OLMo 2's post-norm with q/k norms; Llama with YaRN, K2's FP32 checkpoint rounded to BF16 as an engine loads it), and every maximal span of an OLMo 2 output is looked up in its own training data with

its count and a source document (`llm/olmo_trace.py`): "The capital of Kosovo is Pristina" occurs 224 times, one of them a DCLM page from graphicmaps.com. On the three prompts (OLMo 2, 39 steps of 30 s, 64 GB each) the exact path writes "The municipalities are divided into 10,908 settlements" and both engines "2,994 settlements"; neither number occurs before "settlements" anywhere in the model's training data, so both are made up, differently. Its wrong product ($123456789 \times 987654321 = 12345678987654321$ on every path) comes with an explanation stitched from training phrases ("This is a special case of a more general phenomenon: the", 11 words, 22 times). K2-Think-V2 runs the same nine sequences from disk (39 steps of 93–106 s, 274 GB read per step, none uncertified; the engine rounds 0.01–0.02% of the BF16 outputs and 69–81% of the FP32 logits differently). Both engines agree with each other and part from the exact path: in Albanian at the 8th generated token (exact "Prishtina, qyteti më i madh në vend", the largest city; engines "Prishtina, qytet i vendosur në pjesën qendrore të vendit", in the centre of the country), in English at the 27th ("confirmation or correction?" against "confirmation or maybe they are stating a fact"). Both texts are true; the rounding picks which one. On the product all three paths still reason identically after 30 generated tokens. Ultra's English continuation, looked up in DCLM (filtered Common Crawl, the public web its data was drawn from), has a 9-word span 11 times.

Many tokens at once, still exact. Decoding one token at a time reads the whole model per token; a benchmark needs the model to read it once per batch. `llm/bgemm.cu` makes the batched products exact as well. An FP8 E4M3 weight times 2^9 is an integer below 2^{18}: three balanced base-128 digits, so a 128×128 scale block is nine INT8 MMA chains whose block sums fit an `int64`, added in a certified 128-bit window. For BF16×BF16 each block of 128 shares a grid at its largest bit; weights take 3 digits, tokens 4 (5 and 6 when the output is FP32); elements below the grid are floored, counted and bounded, and every output the bound cannot certify is recomputed exactly on the CUDA cores, so no output is ever wrong. `llm/jais_batch.cu` runs Jais-3-375B-A23B-FP8 (61 layers, 192 experts top-8 in layers 3–60, FP8 experts with BF16 128×128 block scales, 391 GB) layer by layer over many tokens, with the same host functions as the token-by-token driver. On the three prompts the whole model gives the token-by-token run bit for bit: all 126 step, digest and sequence lines identical, 227 160 product rows, 729.6 million BF16 outputs of which 0.035% went to the exact fixup, 30 million FP32 logits of which 2 did, none uncertified. A reader thread keeps the next layers in flight: the prefill of 32 rows falls from 169 to 86 s (162.7 GB in 37.6 s, 4.3 GB/s from NVMe) and decoding from 119 to 29 s per step.

A security benchmark on the exact path. CTIBench's multiple-choice set (CTI-MCQ, 2 500 cyber-threat-intelligence questions) was given to Jais-3 with its official prompt and the model's own chat template. Jais-3 has no mode without reasoning, so the prompt ends with an empty thinking block, as the template writes one. Three protocols were measured on the first chunk. Reading the letter logits right after the empty thinking scores 45.9%, but the greedy token there is "The" in 147 of 157 prompts, so it measures a token the model does not write. Generating shows why: the model reasons in its answer ("The user is asking . . . ") and the letter comes hundreds of tokens later, days for the whole set on this laptop. The reported protocol appends "The answer is" and reads the logits of " A". . . " D": the greedy token is then a letter in every prompt, and Jais-3 answers 1 836 of the 2 500 questions (73.4%; always answering the commonest letter gives 37.1%). A chunk of ∼24 000 rows takes 992–1 060 s (the first 1 396 s), 663–709 s of them in exact products, none uncertified. Moving the attention to the GPU's FP64 units, in the host's order of operations with the softmax left on the host, changes none of its outputs on a whole chunk of 23 995 rows (a run that computes it both ways and compares them); on the GPU the attention takes 41 s of such a chunk. On the root-cause set (CTI-RCM, 1 000 CVE descriptions to be mapped to a CWE) the prompt ends with "CWE-" and five greedy tokens give the number: 738 are right (73.8%; the commonest class, CWE-79, is 22.9%). Neither protocol is the benchmark's official one, which lets the model reason before it answers; the numbers are

reproducible to the bit, not comparable to published scores.

11 Energy

Under these kernels the laptop is power-bound: 94–117 W across our runs, with the SM clock at 1.8–2.2 GHz (2.67–2.76 GHz with only the FP64 units busy). For GEMM at $n = 4096$ (`bench/energy.cu`, 12 s per run, power every 200 ms; three runs each on 3 November with the current code, `bench/energy_all.sh`): native FP64 6.8–7.3 GFLOPS/W at 55–59 W, Ozaki-II at 16 moduli 50.2–51.3 (6.9–7.5×; 5.79–5.87 TFLOPS sustained at 113–117 W), at 12 moduli 64.4–65.1. The code of 15 October gave 7.4, 53.6 and 69.5 at ∼95 W: the current code is 14–16% faster at 16 moduli and draws 20–24% more power, so its joules per product barely moved; under a power cap only fewer operations help (Section 12). For a whole solve ($n = 4096$, $\kappa = 10^{20}$, 20 back-to-back GPU solves after a warm-up; CPU package power from the RAPL counters, GPU power from `nvidia-smi`):

	time	CPU package	GPU	energy per solve
`__float128` LU, 24 threads	55.0 s	106.5 W	8.4 W (idle)	6.3 kJ
dd LU on the GPU	0.238 s	28.2 W	68.3 W	**23 J**

That is ∼270× less energy per solve with both CPU and GPU counted (360× GPU alone against CPU package alone). DRAM power is not exposed on this laptop.

12 Past the classical ceiling: Strassen over $\mathbb{Z}/m$

Every Ozaki-II product is a product of residues modulo $m \le 256$. Strassen's algorithm [5] is exact in any ring, so it applies to every modulus, composite ones included, and its operand sums, reduced modulo m, are again residues: unlike over the integers, no level adds a bit, and every level stays on the INT8 tensor cores. The result is bit-identical to the classical product, so the stability question that keeps fast algorithms out of floating-point emulation does not arise. We count the *effective* rate $2n^3/t$ in classical operations and check every run byte for byte against the classical product (64 of whose entries are recomputed on the CPU, and a changed byte must be counted).

$n = 32\,768$, residues mod 251	time	effective TOPS
classical, 128 × 128 tile	553 ms	127.1
classical, 128 × 256 tile, memory clock 9001 MHz	443–470 ms	149.7–158.7
Strassen, 2 levels, level by level	456 ms	154.1
Strassen, 2 levels in one pass, 128 × 256, 9001 MHz	**375 ms**	**187.8**

Two levels in one pass. Two levels of Strassen are one bilinear algorithm with 49 products whose coefficients are the Kronecker products $U \otimes U$, $V \otimes V$, $W \otimes W$ of the one-level ones. Each of the 16 sub-blocks is read once and each of the 49 operands written once, the 49 products run as one batched launch, and the 16 sub-blocks of C are written once and never read: about $8h^2$ bytes per node of size h instead of $\sim 39h^2$ level by level (at $n = 32\,768$ with three levels, 125.9 → 168.5 effective TOPS).

What the hardware gives. Under this load the GPU draws 100 W and holds 2195 MHz: 46 SMs × 2048 INT8 operations per clock give 206.8 TOPS; the 232 TOPS of the short peak test (Table 1) need ∼2.46 GHz, which the laptop does not sustain. The classical product reaches 72–77% of the 206.8; Strassen modulo m reaches 91%, in classical operations.

The power cap is an energy budget. A pure DRAM copy draws 84 W at 460 GB/s, the GEMM 100–110 W. Running the memory passes on separate streams beside the products, which

use different units, gained nothing (433.7 against 433.2 ms at four levels); lowering the memory clock moved watts to the SMs (classical +10%), and a tile that reads a third fewer shared-memory bytes per multiply-add gave +13% (classical) and +11% (Strassen) at $n = 16\,384$. Under a power cap time is energy divided by power: only fewer joules help. At four levels the 2 401 products alone take 272–288 ms, 244–259 effective TOPS, above the 232 TOPS of that test; the memory passes (∼180 ms) take it back to 164. Fusing the operand sums into the products' loads is the next step.

Inside Ozaki-II, measured in joules. Running every modular product of the FP64 emulation through the two-level pass (any modulus $m \le 256$: the reduction $v - m\,\mathrm{rint}(v/m)$ in single precision is exact for $|v| < 2^{21}$) leaves the residue planes of C byte-identical to the classical ones (0 of $3.76 \cdot 10^9$ bytes differ at $n = 16\,384$ with 14 moduli), hence every FP64 entry. A second change saves a modulus: scaling each row by its 2-norm instead of its maximum, the Cauchy–Schwarz bound $|C'_{ij}| \le \|a'_i\|\,\|b'_j\| < M/2$ (M the product of the moduli, Section 2) holds with $\beta = \lfloor(\log_2 M - 1.001)/2\rfloor$ bits for each row norm, independently of K, and a row spread over K entries keeps $\log_2(\|a_i\|/\max_k |a_{ik}|)$ more bits (∼3 at $K = 16\,384$), so 14 moduli keep FP64 accuracy (2.03 units against the native 2.82 below), which with the maximum takes 15. Energy is the integral of NVML power samples over at least 8 s of repeated products; clocks are set with `nvidia-smi`. With $n = 16\,384$, $C = AB^{\mathsf{T}}$, Gaussian entries, and the error the maximum over 1 024 sampled entries of $|C - C_{\mathrm{ref}}|/(|A||B^{\mathsf{T}}|)$ in units of 2^{-53} against a double-double reference:

	SM/memory MHz	time	J per DGEMM	error
native FP64	default	17.32 s	1 441.5	2.82
native FP64, its best point	1500/9001	32.17 s	1 025.7	2.82
Ozaki-II, 15 moduli, classical	default	1.009 s	119.4	0.98
+ 2-norm scale, 14 moduli	default	0.942 s	114.4	2.03
+ Strassen, two levels in one pass	default	0.854 s	103.3	2.03
+ operating point	**1350/9001**	1.376 s	**73.6**	**2.03**

That is 119.4 GFLOPS/W at FP64 accuracy: 19.6× fewer joules than the GPU's own FP64 at default clocks, 13.9× fewer than at its best operating point, 20.3× faster at default clocks, and more accurate (6.89 against 17.14 with magnitudes spread over $2^{\pm 8}$). The GPU idles at ∼17 W: the best point is 23.5 J static and 50.1 J dynamic, the native FP64 at its best point 557 J static and 469 J dynamic, so the dynamic joules alone are 9.4× fewer. Below ∼1350 MHz the fixed power dominates and the joules rise again. Residues of A for all moduli come from one read of A into the planes of C (the result of modulus l overwrites plane l), and the reconstruction ends in a 128-bit integer Horner scheme, one rounding instead of 13 at the FP64 rate. At this size the 12 GB card still forces operand chunks of 23 of the 49, so Strassen saves 9% of the time and 10% of the joules here (default clocks), against 20% of the time for the bare product. For scale only (rated peak over rated power, not measured): an H100 SXM lists 67 TFLOPS of FP64 tensor-core arithmetic at up to 700 W (96 GFLOPS/W), an H100 NVL 60 TFLOPS at 350–400 W (150–171).

13 Limitations

The INT8 GEMMs are power-bound on this laptop, so hiding latency does not translate into time; gains come from fewer operations (fewer moduli, Strassen's seven products for eight) and less data movement, including inside the kernel (Section 12). Strassen pays from $n \approx 16\,384$ here, where 16 residue planes and the FP64 operands no longer fit 12 GB together; inside the FP64 emulation the card still forces operand chunks, so it saves 9% of the time at $n = 16\,384$. The cost of the higher precisions grows roughly with the square of the number of slices, as MPFR's does at these sizes, so the ratio against MPFR does not grow without bound; it is largest against software emulation of each width. The triple-double solver is panel-bound at $n = 4096$. Kernel

launches cost ∼7 μs under WSL2 (0.83 μs per node in a CUDA graph), which matters for the many small launches of the panel. For FP64 itself our GEMM is not the one to use: on this laptop cuBLAS 13's emulation and GEMMul8 are faster and at least as accurate (Section 5). The CPU baselines of the double-double and triple-double solves are our own LUs in the same arithmetic, cache-blocked (AVX2, 24 threads, bit for bit the unblocked loop); against those the GPU's lead is 22× and 9×, not the hundreds it has against `__float128` and MPFR, and at $n \leq 2048$ it is small. We have not measured MPLAPACK's double-double LU. All measurements are on one laptop model; the CPU references use the same machine's 24 threads.

14 Related work

Ozaki scheme II [12] emulates FP64 GEMM with INT8 and the CRT and reports 7.4–9.8 TFLOPS on an RTX 4090; RIKEN's GEMMul8 [17] is its public code. NVIDIA's cuBLAS has emulated DGEMM on INT8 tensor cores since CUDA Toolkit 13.0 Update 2 (October 2025), by the Ozaki scheme with fixed-point slices [13], either with an automatic dynamic precision that analyses the inputs and keeps an accuracy equal to or better than native FP64 (or runs native FP64 instead) or with a fixed number of mantissa bits [15, 16]; NVIDIA reports up to 13× native FP64 on an RTX PRO 6000 Blackwell Server Edition. Both return an FP64 result at most; Section 5 measures them beside ours on this laptop. Follow-up work covers SGEMM and power efficiency on an RTX 5080 [18], FP8 and FP4 variants [19, 20, 21], fused kernels reaching 81% of INT8 peak on Blackwell [22] and complex GEMM [23]. Beyond FP64, Ozaki scheme I already gave binary128 GEMM from FP64 GEMMs [6], and Kouya runs Ozaki-II for multiple-precision GEMM, 53 to 2048 bits, on CPU INT8 units (Intel AMX, Arm SVE2-i8mm) [25]; Chen and Verschelde multiply multiple doubles on the FP64 tensor cores of an A100 with an Ozaki-like splitting [14]; we found no double-double or wider GEMM built from INT8 tensor cores, the only fast ones consumer and laptop GPUs have, nor linear solves at these condition numbers on this class of hardware. Multimodular exact integer matrix multiplication is classical (FLINT; conversions to and from a residue system written as matrix products by Doliskani, Giorgi, Lebreton and Schost [24]). Close in spirit: integer matrix products at various precisions on AMX and IFMA by van der Hoeven and Mezzarobba [26]; modular products over large prime fields on CPUs and GPUs by Berthomieu, Graillat, Lesnoff and Mary [27]; EmuGEMM already fuses the modular reduction into the GEMM epilogue [22]. We have not found exact integer GEMM of 10^3–10^5-bit entries on GPU INT8 tensor cores, conversions included, measured against FLINT; byte-split residues are what make it run there. For low-precision neural networks, exact (Kulisch-style) accumulation was proposed in hardware designs [2, 3]; batch-invariant kernels [1] make LLM inference deterministic across batch sizes on one setup, and say explicitly that they are not hardware or software-version invariant. Correct rounding of every product, as in Section 10, is invariant to both, on commodity INT8 units.

15 Conclusion

On a laptop GPU whose FP64 units are 517 times slower than its INT8 tensor cores, precision becomes a software parameter: the same INT8 GEMMs yield FP64, double-double, triple-double and wider products, and an ill-conditioned dense solve that needs quad precision finishes 22× sooner than the same LU cache-blocked on the laptop's 24 CPU threads and 629× sooner than `__float128` ($n = 8192$), with ∼270× less energy than `__float128` ($n = 4096$). Exact integer products of up to 131 063-bit entries come from the same units, 10–19× faster than FLINT. Exact determinants and exact rational solutions of int8 systems, where no big number has to enter the GPU, come 36–355× faster than FLINT, and with 32–256-bit entries high-order lifting on the same GEMMs keeps the ratio rising with the bits, up to 404×. On real matrices of NIST Matrix Market the certified exact solution comes 9–7 857× sooner than SPEX, where FP64 often has no

correct digit; and the same units give correctly rounded MXFP4, NVFP4, FP8 and BF16 products on the real weights of open models of up to 2.78 trillion parameters, identical for any batch, and run the whole 560-billion-parameter Nemotron 3 Ultra from disk that way at no cost in decode, where the disk sets the pace; batched over many tokens, Jais-3-375B gives its token-by-token run bit for bit. Under the laptop's power cap only fewer joules help: Strassen's algorithm over $\mathbb{Z}/m$, exact for every modulus, reaches 91% of the sustained INT8 rate, and with a 2-norm scaling that saves a modulus, at the GPU's most efficient clock, it gives an FP64-accurate DGEMM for 73.6 J at $n = 16\,384$, 13.9× fewer joules than the GPU's own FP64 at its best clock. The next steps are the panel (now the bound of the triple-double solve), CUDA graphs for its launches, a subquadratic residue conversion for wide integers, the operand sums of Strassen fused into the products' loads, and out-of-core sizes beyond the GPU's 12 GB.

References

[1] H. He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Blog post, 10 September 2025, https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference.

[2] J. Johnson. Rethinking floating point for deep learning. arXiv:1811.01721, 2018.

[3] Z. Carmichael et al. Deep Positron: a deep neural network using the posit number system. arXiv:1812.01762, 2018.

[4] C. Lourenco, J. Chen, E. Moreno-Centeno, T. A. Davis. Algorithm 1021: SPEX Left LU, exactly solving sparse linear systems via a sparse left-looking integer-preserving LU factorization. *ACM Trans. Math. Softw.* 48(2):1–23, 2022. doi:10.1145/3519024.

[5] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik* 13(4):354–356, 1969. doi:10.1007/BF02165411.

[6] D. Mukunoki, K. Ozaki, T. Ogita, T. Imamura. Accurate matrix multiplication on binary128 format accelerated by Ozaki scheme. Proc. 50th International Conference on Parallel Processing (ICPP), 1–11, 2021. doi:10.1145/3472456.3472493.

[7] J. D. Dixon. Exact solution of linear equations using p-adic expansions. *Numerische Mathematik* 40(1):137–141, 1982. doi:10.1007/BF01459082.

[8] S. Toledo. Locality of reference in LU decomposition with partial pivoting. *SIAM J. Matrix Anal. Appl.* 18(4):1065–1081, 1997. doi:10.1137/S0895479896297744.

[9] E. Kaltofen, B. D. Saunders. On Wiedemann's method of solving sparse linear systems. Proc. AAECC-9, LNCS 539, 29–38, Springer, 1991. doi:10.1007/3-540-54522-0_93.

[10] V. Pan. Computing the determinant and the characteristic polynomial of a matrix via solving linear systems of equations. *Information Processing Letters* 28(2):71–75, 1988. doi:10.1016/0020-0190(88)90166-4.

[11] J. Abbott, M. Bronstein, T. Mulders. Fast deterministic computation of determinants of dense matrices. Proc. ISSAC 1999, 197–204. doi:10.1145/309831.309934.

[12] K. Ozaki, Y. Uchino, T. Imamura. Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique. arXiv:2504.08009, 2025.

[13] A. Schwarz, A. Anders, C. Brower, H. Bayraktar, J. Gunnels, K. Clark, R. G. Xu, S. Rodriguez, S. Cayrols, P. Tabaszewski, V. Podlozhnyuk. Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme. Proc. SCA/HPCAsia 2026, Osaka. doi:10.1145/3773656.3773670.

[14] H. Chen, J. Verschelde. Multiple double arithmetic on NVIDIA tensor cores. arXiv:2607.06881, 2026 (ICMS 2026).

[15] C. Brower, J. Gunnels, G. Lopez. Unlocking Tensor Core performance with floating point emulation in cuBLAS. NVIDIA Technical Blog, 24 October 2025, https://developer.nvidia.com/blog/unlocking-tensor-core-performance-with-floating-point-emulation-in-cublas/.

[16] NVIDIA. cuBLAS library documentation, cuBLAS 13: floating point emulation (`CUBLAS_COMPUTE_64F_EMULATED_FIXEDPOINT`), https://docs.nvidia.com/cuda/cublas/.

[17] RIKEN R-CCS. GEMMul8: high-precision BLAS-like matrix operations emulated with INT8/FP8 matrix engines by the Ozaki scheme II. MIT license, https://github.com/RIKEN-RCCS/GEMMul8 (commit 30b5b5d, 3 November 2026).

[18] Y. Uchino, K. Ozaki, T. Imamura. High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines. arXiv:2508.03984, 2025.

[19] Y. Uchino, K. Ozaki, T. Imamura. Double-precision matrix multiplication emulation via Ozaki-II scheme with FP8 quantization. arXiv:2603.10634, 2026.

[20] S. Hayashi, D. Mukunoki, T. Hoshino, T. Katagiri. DGEMM with Ozaki Scheme I/II on FP4 tensor cores: a base-13 E2M1 limb representation. arXiv:2608.06812, 2026.

[21] S. Hayashi, D. Mukunoki, T. Hoshino, T. Katagiri. AWE: Adaptive weight encoding for exact integer matrix products with fewer GEMMs on FP4 tensor cores. arXiv:2609.24519, 2026.

[22] D. Lu, A. Maeder, M. Luisier, A. N. Ziogas. EmuGEMM: Fused tensor core kernels for precision emulation in matrix multiplication. arXiv:2606.25453, 2026.

[23] Y. Uchino, Q. Ma, T. Imamura, K. Ozaki, P. L. Gutsche. Emulation of complex matrix multiplication based on the Chinese remainder theorem. arXiv:2512.08321, 2025.

[24] J. Doliskani, P. Giorgi, R. Lebreton, É. Schost. Simultaneous conversions with the residue number system using linear algebra. ACM Trans. Math. Softw. 44(3):1–21, 2018. doi:10.1145/3145573.

[25] T. Kouya. Ozaki Scheme II is fast on CPUs too: multiple-precision matrix multiplication on Intel AMX-INT8 and Arm SVE2-i8mm. arXiv:2609.27831, 2026.

[26] J. van der Hoeven, M. Mezzarobba. Integer matrix multiplication at various precisions using AMX and IFMA. Preliminary version of June 1, 2026, https://www.texmacs.org/joris/amx/amx.html.

[27] J. Berthomieu, S. Graillat, D. Lesnoff, T. Mary. Multiword matrix multiplication over large finite fields in floating-point arithmetic. arXiv:2601.07508, 2026.

[28] T. Ogita. Accurate and verified numerical computation of the matrix determinant. Int. J. Reliability and Safety 6(1/2/3) (2012) 242. doi:10.1504/IJRS.2012.044287.

[29] A. Urbańska. Towards an exact adaptive algorithm for the determinant of a rational matrix. arXiv:0706.0014, 2007.

[30] A. Storjohann. The shifted number system for fast linear algebra on integer matrices. J. Complexity 21(4):609–650, 2005. doi:10.1016/j.jco.2005.04.002.

[31] C. Pauderis, A. Storjohann. Deterministic unimodularity certification. Proc. ISSAC 2012, 281–288. doi:10.1145/2442829.2442870.

[32] NVIDIA. NVIDIA-Nemotron-3-Ultra-550B-A55B (model card; BF16 and NVFP4 checkpoints). Hugging Face, 3 June 2026, https://huggingface.co/nvidia/NVIDIA-Nemotron-3-Ultra-550B-A55B-NVFP4.